MLPy Workshop 5: Solutions

February 18, 2026

1 Week 5 - Regularization

Student Name 1, Student Name 2

1.1 Aims

By the end of this notebook you will be able to

- perform regulized regression in sklearn
- understand the role of tuning parameter(s)
- use cross-validation for model tuning and comparison.

1. Problem Definition and Setup
2. Exploratory Data Analysis
3. Baseline Model
4. Ridge Regression
5. Lasso Regression
6. ElasticNet Regression

During workshops, you will complete the worksheets together in teams of 2-3, using **pair programming**. You should aim to switch roles between driver and navigator approximately every 15 minutes. When completing worksheets:

- You will have tasks tagged by (CORE) and (EXTRA).
- Your primary aim is to complete the (CORE) components during the WS session, afterwards you can try to complete the (EXTRA) tasks for your self-learning process.

Instructions for submitting your workshops can be found at the end of worksheet. As a reminder, you must submit a pdf of your notebook on Learn by 16:00 PM on the Friday of the week the workshop was given.

---

2 Problem Definition and Setup

2.1 Packages

First, let's load some of the packages you wil need for this workshop (we will load others as we progress).

[1]:
```python
# Data libraries
import pandas as pd
import numpy as np

# Plotting libraries
import matplotlib.pyplot as plt
import seaborn as sns

# sklearn modules
import sklearn
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import GridSearchCV, KFold
```

2.2 User Defined Helper Functions

We will make use of the two helper functions. The first is identical to the one last week. The second one extract the coefficients and names (if required), plots the coefficients and returns a data frame.

[2]:
```python
def model_fit(m, X, y, plot = False):
    """Returns the mean squared error, root mean squared error and R^2 value of␣
↪a fitted model based
    on provided X and y values.

    Args:
        m: sklearn model object
        X: model matrix to use for prediction
        y: outcome vector to use to calculating rmse and residuals
        plot: boolean value, should residual plots be shown
    """

    y_hat = m.predict(X)
    MSE = mean_squared_error(y, y_hat)
    RMSE = np.sqrt(mean_squared_error(y, y_hat))
    Rsqr = r2_score(y, y_hat)

    Metrics = (round(MSE, 4), round(RMSE, 4), round(Rsqr, 4))

    res = pd.DataFrame(
        data = {'y': y, 'y_hat': y_hat, 'resid': y - y_hat}
    )

    if plot:
        plt.figure(figsize=(12, 6))

        plt.subplot(121)
```

```
        sns.lineplot(x='y', y='y_hat', color="grey", data =  pd.
↪DataFrame(data={'y': [min(y),max(y)], 'y_hat': [min(y),max(y)]}))
        sns.scatterplot(x='y', y='y_hat', data=res).set_title("Observed vs␣
↪Fitted values")

        plt.subplot(122)
        sns.scatterplot(x='y_hat', y='resid', data=res).set_title("Fitted␣
↪values vs Residuals")
        plt.hlines(y=0, xmin=np.min(y), xmax=np.max(y), linestyles='dashed',␣
↪alpha=0.3, colors="black")

        plt.subplots_adjust(left=0.0)

        plt.suptitle("Model (MSE, RMSE, Rsq) = " + str(Metrics), fontsize=14)
        plt.show()

    return MSE, RMSE, Rsqr
```

[3]:

```
def get_coefs(m, plot = False, feature_names = None, figsize = (5,5), figtitle␣
↪= None, intercept = True):
    """Returns model coefficients in a data frame for a fitted linear model.

    Args:
        m: sklearn LinearRegression model object or pipeline with␣
↪LinearRegression as final step
        plot: boolean value, should coefficients be plotted with error bars
        feature_names: list of feature names to use in the plot
        figsize: tuple defining figure size
        figtitle: string defining figure title
        intercept: boolean value, should intercept be included in the plot
    """

    # Extract intercept and coefficients into a single array
    w = np.concatenate(([m[-1].intercept_] if isinstance(m, sklearn.pipeline.
↪Pipeline) else [m.intercept_],
                        m[-1].coef_ if isinstance(m, sklearn.pipeline.
↪Pipeline) else m.coef_))
    # Extract name of features
    if feature_names is None:
        feature_names = m[:-1].get_feature_names_out() if isinstance(m, sklearn.
↪pipeline.Pipeline) else m.feature_names_in_
    feature_names = np.concatenate((['intercept'], feature_names))
    # Create a data frame
    w_df = pd.DataFrame({'feature': feature_names, 'coef': w}).sort_values␣
↪("coef", ascending=False)
```

```
    if plot:
        if not intercept:
            w_df = w_df[w_df['feature'] != 'intercept']
        plt.figure(figsize=figsize)
        plt.barh(w_df['feature'], w_df['coef'])
        plt.ylabel('Features')
        plt.xlabel('Coefficient Value')
        plt.axvline(x=0, color=".5")
        if figtitle is not None:
            plt.title(figtitle)
        plt.grid()
        plt.show()

    return  w_df
```

2.3 Data

The data for this week's workshop comes from the Elements of Statistical Learning textbook. The data originally come from a study by Stamey et al. (1989) in which they examined the relationship between the level of prostate-specific antigen (`psa`) and a number of clinical measures in men who were about to receive a prostatectomy. The variables are as follows,

- `lpsa` - log of the level of prostate-specific antigen
- `lcavol` - log cancer volume
- `lweight` - log prostate weight
- `age` - patient age
- `lbph` - log of the amount of benign prostatic hyperplasia
- `svi` - seminal vesicle invasion
- `lcp` - log of capsular penetration
- `gleason` - Gleason score
- `pgg45` - percent of Gleason scores 4 or 5
- `train` - test / train split used in ESL

These data are available in `prostate.csv`, which is included in the workshop materials.

Let's start by reading in the data.

[4]:
```
prostate = pd.read_csv('prostate.csv')
prostate.head()
```

[4]:
```
     lcavol   lweight  age      lbph  svi       lcp  gleason  pgg45      lpsa  \
0 -0.579818  2.769459   50 -1.386294    0 -1.386294        6      0 -0.430783
1 -0.994252  3.319626   58 -1.386294    0 -1.386294        6      0 -0.162519
2 -0.510826  2.691243   74 -1.386294    0 -1.386294        7     20 -0.162519
3 -1.203973  3.282789   58 -1.386294    0 -1.386294        6      0 -0.162519
4  0.751416  3.432373   62 -1.386294    0 -1.386294        6      0  0.371564

  train
0     T
```

```
1       T
2       T
3       T
4       T
```

3 Exploratory Data Analysis

Before modelling, we will start with EDA to gain an understanding of the data, through descriptive statistics and visualizations.

3.0.1 Exercise 1 (CORE)

a) Examine the data structure, look at the descriptive statistics, and create a pairs plot. Do any of our variables appear to be categorical / ordinal rather than numeric?

b) Are there any interesting patterns in these data? Which variable appears likely to have the strongest relationship with `lpsa`? Why do you think we are exploring the relationship between these variables and `lpsa` (log of psa) rather than just psa?

[5]:
```python
# For general info
prostate.info()

# For the summary statistics
prostate.describe()
```

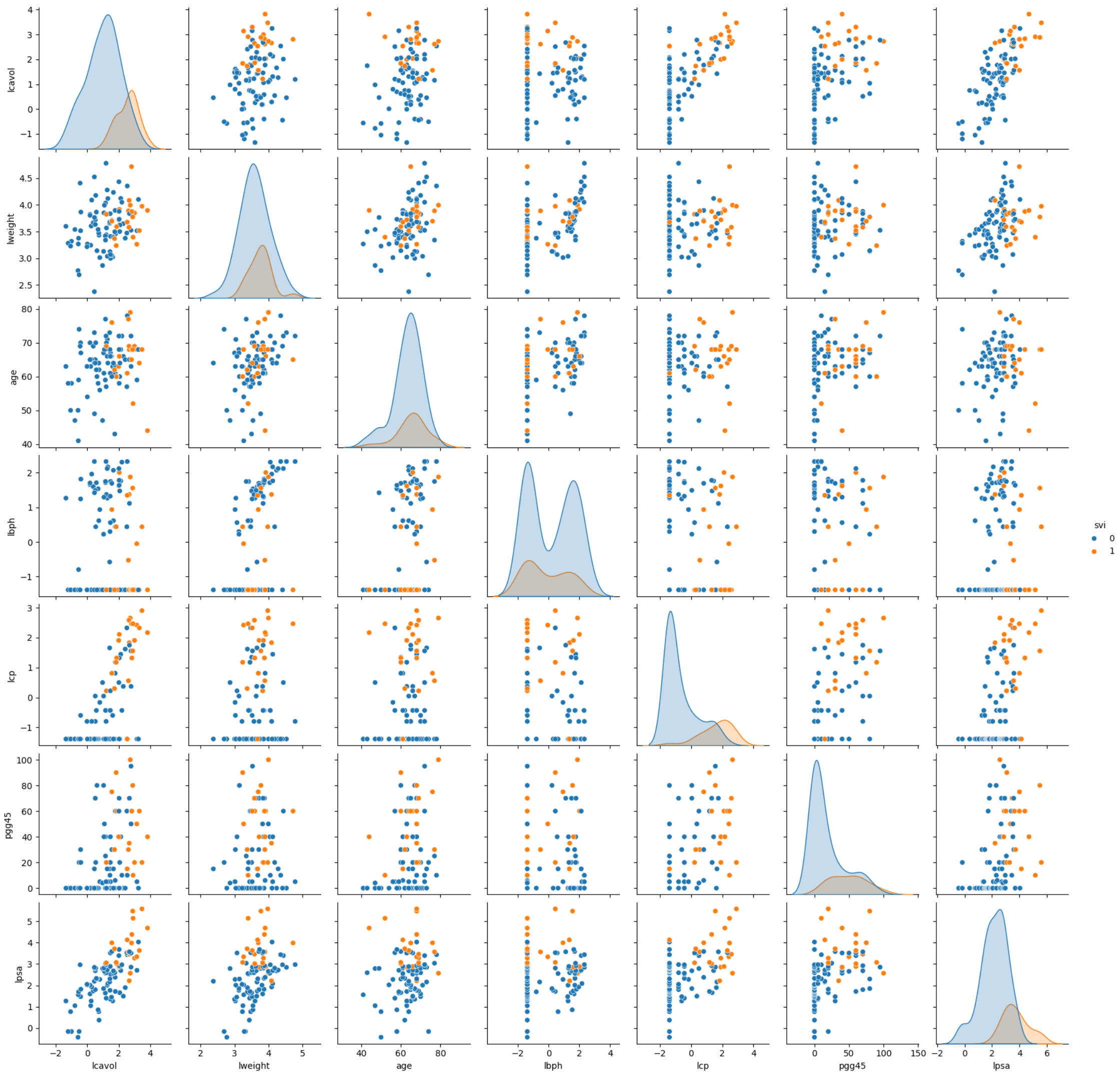
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 97 entries, 0 to 96
Data columns (total 10 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   lcavol   97 non-null     float64
 1   lweight  97 non-null     float64
 2   age      97 non-null     int64
 3   lbph     97 non-null     float64
 4   svi      97 non-null     int64
 5   lcp      97 non-null     float64
 6   gleason  97 non-null     int64
 7   pgg45    97 non-null     int64
 8   lpsa     97 non-null     float64
 9   train    97 non-null     object
dtypes: float64(5), int64(4), object(1)
memory usage: 7.7+ KB
```

[5]:
```
          lcavol    lweight        age       lbph        svi        lcp  \
count  97.000000  97.000000  97.000000  97.000000  97.000000  97.000000
mean    1.350010   3.628943  63.865979   0.100356   0.216495  -0.179366
std     1.178625   0.428411   7.445117   1.450807   0.413995   1.398250
min    -1.347074   2.374906  41.000000  -1.386294   0.000000  -1.386294
```

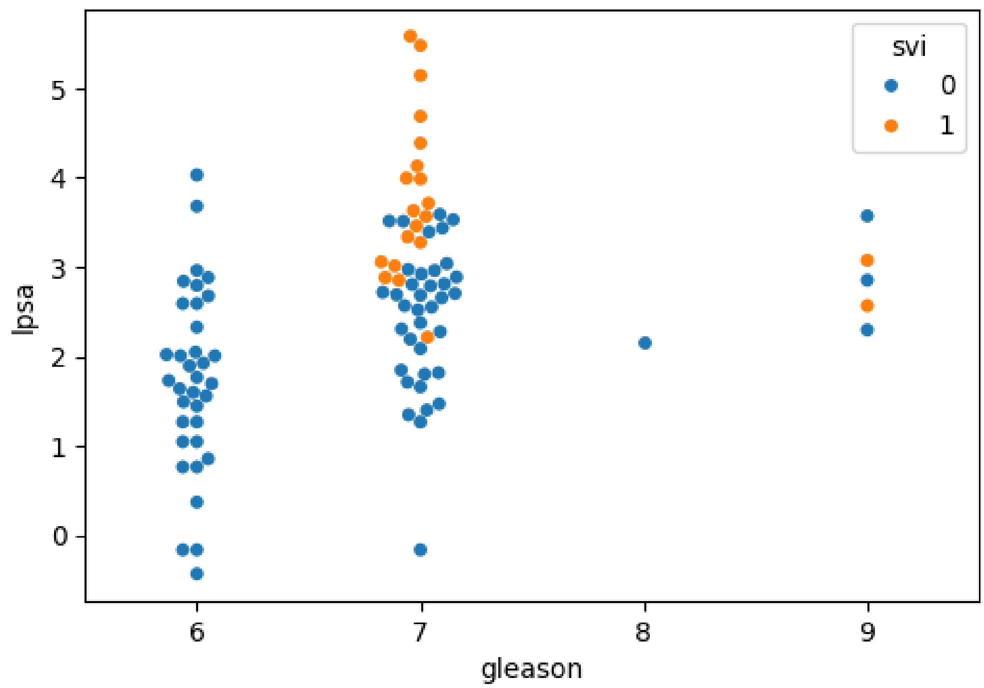
25%	0.512824	3.375880	60.000000	-1.386294	0.000000	-1.386294
50%	1.446919	3.623007	65.000000	0.300105	0.000000	-0.798508
75%	2.127041	3.876396	68.000000	1.558145	0.000000	1.178655
max	3.821004	4.780383	79.000000	2.326302	1.000000	2.904165

	gleason	pgg45	lpsa
count	97.000000	97.000000	97.000000
mean	6.752577	24.381443	2.478387
std	0.722134	28.204035	1.154329
min	6.000000	0.000000	-0.430783
25%	6.000000	0.000000	1.731656
50%	7.000000	15.000000	2.591516
75%	7.000000	40.000000	3.056357
max	9.000000	100.000000	5.582932

[6]:
```
# Initial plot revals that SVI is binary and gleason only takes a small number of values, thus we will exclude from the plot
# sns.pairplot(prostate)
# plt.show()

sns.pairplot(prostate, vars= ['lcavol','lweight','age', 'lbph','lcp', 'pgg45','lpsa'], hue = 'svi')
plt.show()
```




```
[7]: # Better visualization for ordinal categorical variables, svi and gleason
plt.figure(figsize=(6,4))
sns.swarmplot(prostate, x='gleason', y='lpsa', hue = 'svi')
plt.show()
```



There are $N = 97$ observations with no missingness.

- The variables `age` and `ppg45` are integer valued, with latter representing a percent and restricted to the range $[0, 100]$.
- The variable `svi` is binary and the variable `gleason` is and ordered categorical variable
- The other variables are numerical.

We see that both `lcavol` and `lcp` seem to have a linear relationship with the response `lpsa`. Individuals with `svi` seem to have higher `lpsa`.

The response `lpsa` is the log of psa, and this transformation has been applied because on the original scale `psa` is very right skewed, and the log transformation gives a symmetric and more "normal" distribution.

3.1 Train-Test Set

For these data we have already been provided a column to indicate which values should be used for the training set and which for the test set. This is encoded by the values in the `train` column - we can use these columns to separate our data and generate our training data: `X_train` and `y_train` as well as our test data `X_test` and `y_test`.

[8]:
```
# Create train and test data frames
train = prostate.query("train == 'T'").drop('train', axis=1)
test = prostate.query("train == 'F'").drop('train', axis=1)
```

[9]:
```
# Training data
X_train = train.drop(['lpsa'], axis=1)
y_train = train.lpsa

print('X_train:', X_train.shape)
print('y_train:', y_train.shape)
```

```
X_train: (67, 8)
y_train: (67,)
```

[10]:
```
# Test data
X_test = test.drop('lpsa', axis=1)
y_test = test.lpsa

print("X_test:", X_test.shape)
print("y_test:", y_test.shape)
```

```
X_test: (30, 8)
y_test: (30,)
```

Let's also fix the random seed to make this notebook's output identical at every run

[11]:
```
# Fix seed
rng = np.random.seed(0)
```

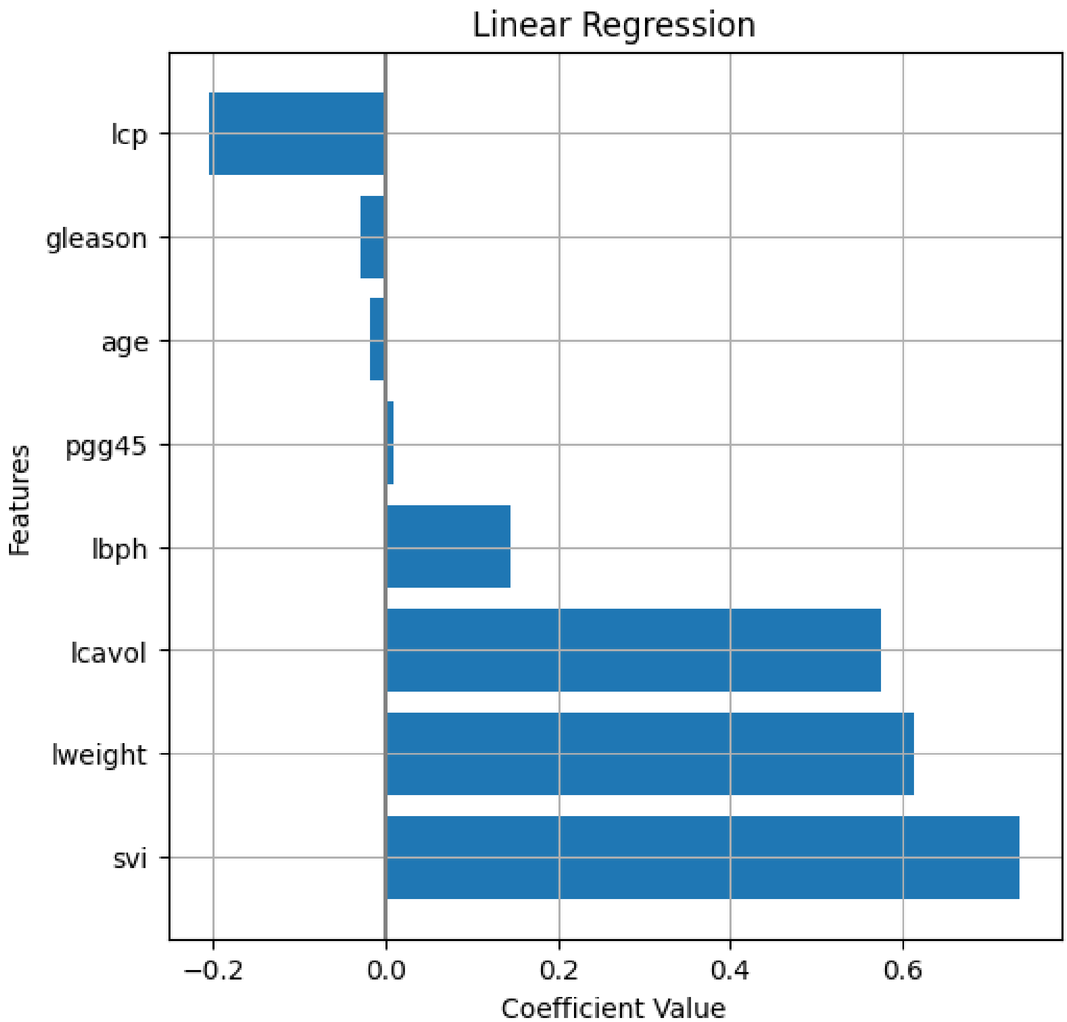
4 Baseline model

Our first task is to fit a baseline model which we will be able to use as a point of comparison for our subsequent models. A good candidate for this is a simple linear regression model that includes all of our features.

[12]:
```
# Train a linear regression model
from sklearn.linear_model import LinearRegression
lm = LinearRegression().fit(X_train, y_train)
```

Using our helper function, we can extract the coefficients for the model, which correspond to the variables: `lcavol`, `lweight`, `age`, `lbph`, `svi`, `lcp`, `gleason`, and `pgg45` respectively.

[13]:
```
w_df = get_coefs(lm, plot=True, figsize=(6,6), figtitle="Linear Regression",␣
 ↪intercept=False)
```



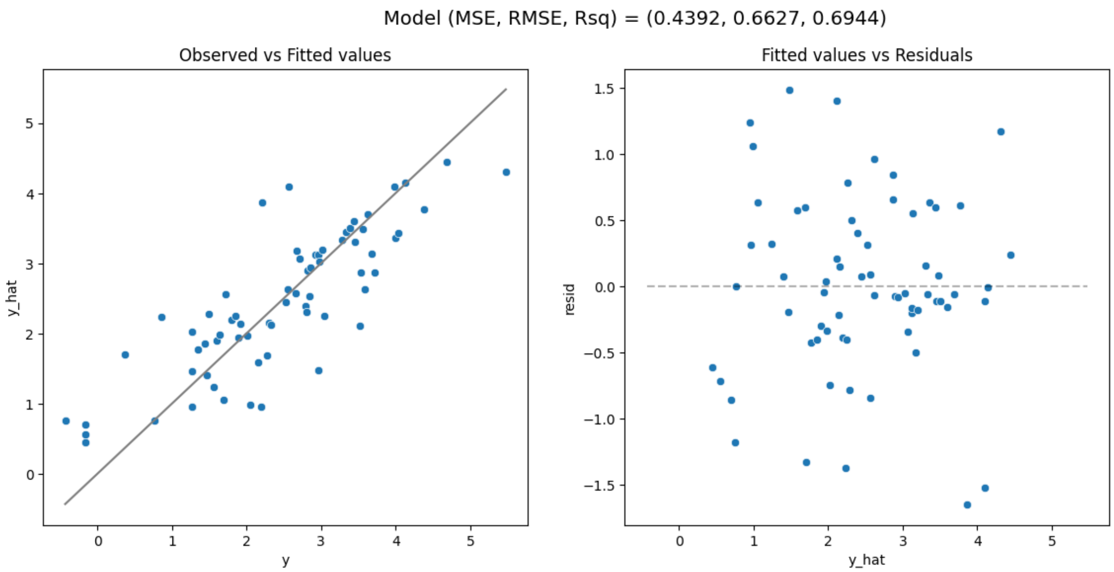
These coefficients have the typical regression interpretation, e.g. for each unit increase in `lcavol` we expect `lpsa` to increase by 0.5765 on average. To evaluate the predictive properities of our model, we will use the `model_fit` helper function.

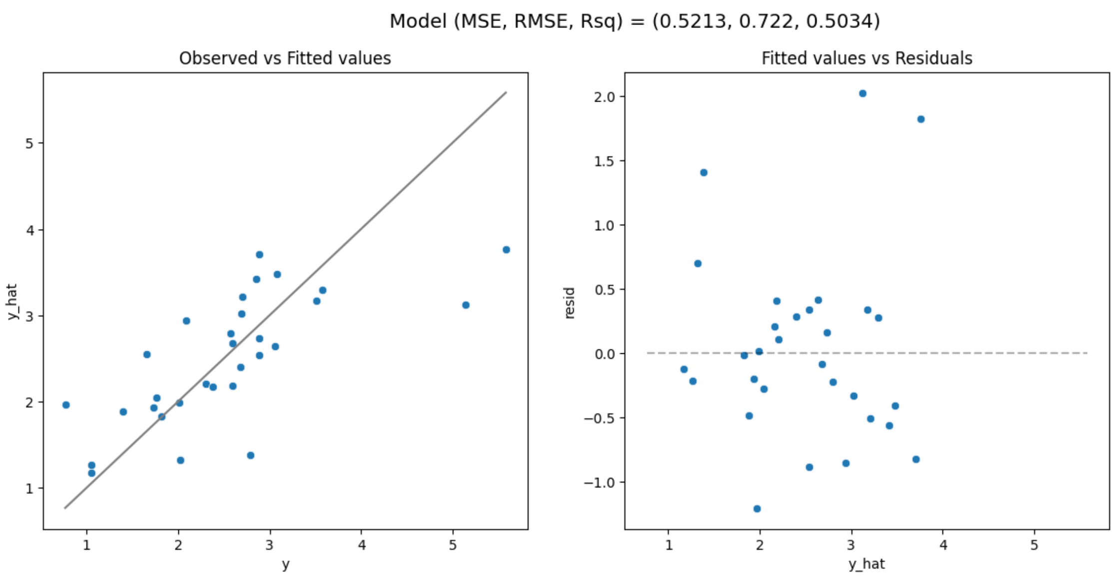
4.0.1 Exercise 2 (CORE)

Use the `model_fit` function to evaluate both the model fit on the training data and the predictions on the test data.

- Based on these plots do you see anything in the fit or residual plot that is potentially concerning?
- Do you expect the MSE on test data to be better or worse than the MSE on the training data?

[14]:
```
lm_metrics_train = model_fit(lm, X_train, y_train, plot=True)
lm_metrics_test = model_fit(lm, X_test, y_test, plot=True)
```





- The model appears to be having difficulties with the smallest and largest values of `lpsa`, i.e. small values are over-predicted (negative residuals) while larger values are underpredicted (positive residuals).
- Generally, the test data have worse MSE than the training data since fitting the least squares estimate is equivalent to minimizing the MSE. The training fit has a better MSE than our testing fit.

[15]:
```
print('Training MSE:',lm_metrics_train[0])
print('Test MSE:',lm_metrics_test[0])
```

```
Training MSE: 0.43919976805833433
Test MSE: 0.5212740055076025
```

4.1 Standardization

In subsequent sections we will be exploring the use of the Ridge and Lasso regression models which both penalize larger values of $\mathbf{w}$. While not particularly bad, our baseline model had coefficients that ranged from the smallest at 0.0095 to the largest at 0.737 which is about a 78x difference in magnitude. This difference can be made even worse if we were to change the units of one of our features, e.g. changing a measurement in kg to grams would change that coefficient by 1000 which has no effect on the fit of our linear regression model (predictions and other coefficients would be unchanged) but would have a meaningful impact on the estimates given by a Ridge or Lasso regression model, since that coefficient would now dominate the penalty term.

To deal with this issue, the standard approach is to standaridize all features. Additionally, the feature values can now be interpreted as the number of standard deviations each observation is away from that column's mean. Using `sklearn` we can perform this transformation using the `StandardScaler` transformer from the preprocessing submodule.

Keep in mind, that in order to avoid **data leakage** and get a realistic idea of the performance of model on the test data, **the mean and standard deviation used to standardize both the training and test sets should be computed from the training data only**. The best way to accomplish this is to include the StandardScaler in a modeling pipeline for your data

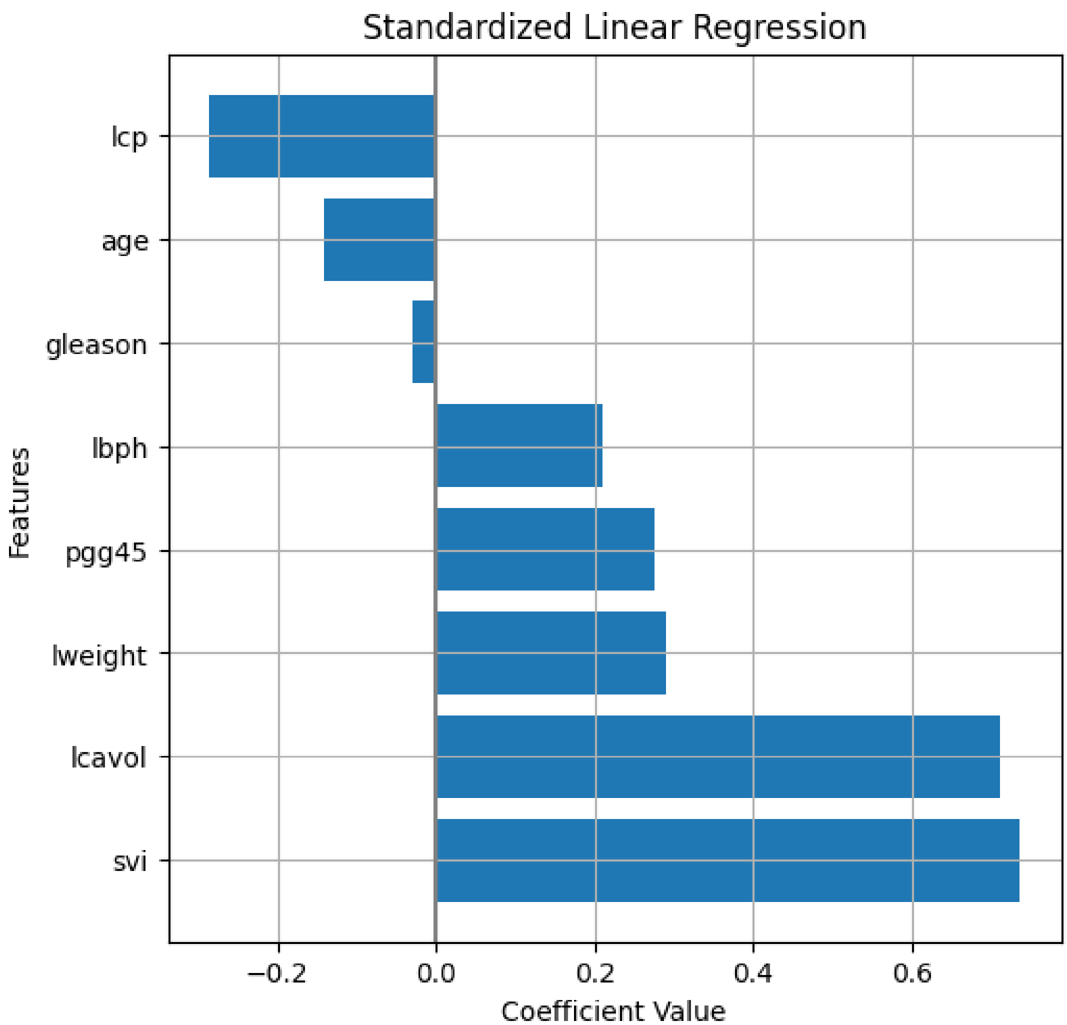
4.1.1 Exercise 3 (CORE)

Consider the following pipeline that first standardizes the numeric features before linear regression. Fit the model to the training data. Using this new model what has changed about our model results? Comment on both the model's coefficients as well as its predictive performance. How has the interpretation of coefficients changed?

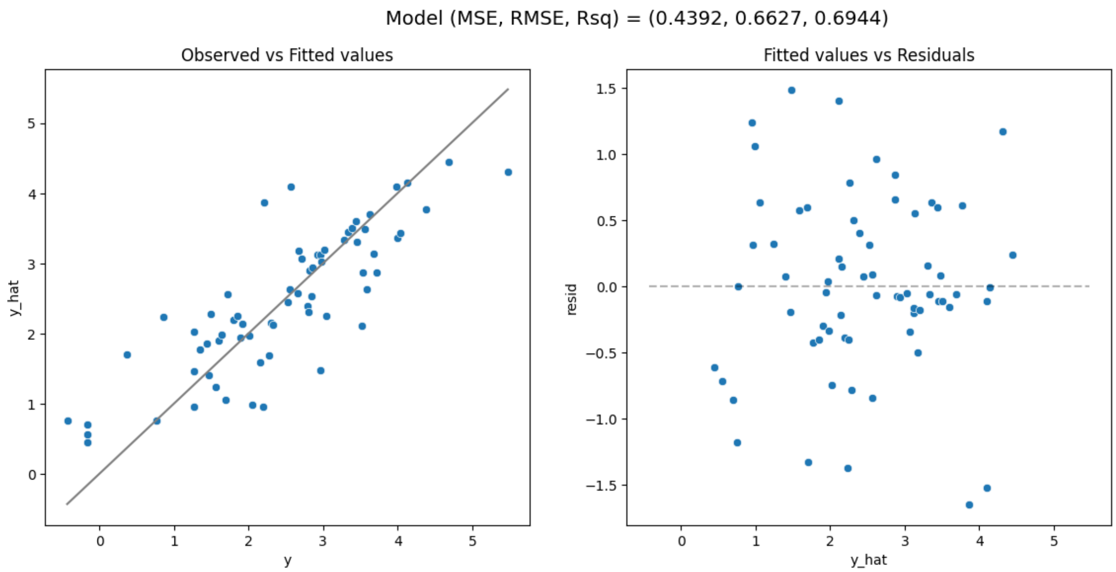
[16]:
```
# Linear regression pipeline, including standardization
from sklearn.compose import make_column_transformer
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OrdinalEncoder

# Names of numeric features
num_features = ['lcavol','lweight','age', 'lbph','lcp', 'pgg45']
# Names of binary features
bin_features = ['svi']
ord_features = ['gleason']

preprocessor = make_column_transformer(
    (StandardScaler(), num_features),
    (OrdinalEncoder(categories=[[6,7,8,9]]),ord_features),
    ('passthrough', bin_features),
    verbose_feature_names_out=False
)
```

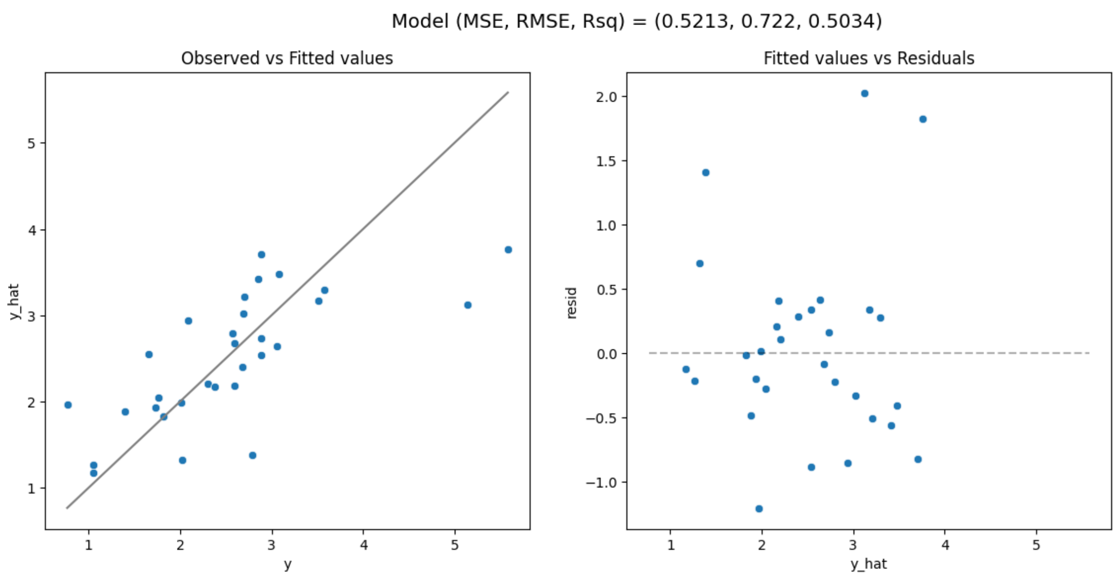
```
lm_s = make_pipeline(
    preprocessor,
    LinearRegression()
).fit(X_train, y_train)
```

[17]:
```
w_df_s = get_coefs(lm_s, plot=True, figsize=(6,6), figtitle="Standardized␣
↪Linear Regression", intercept=False)
```



[18]:
```
lms_metrics_train = model_fit(lm_s, X_train, y_train, plot=True)
lms_metrics_test = model_fit(lm_s, X_test, y_test, plot=True)
```





[19]:
```
print('Training MSE:',lms_metrics_train[0])
print('Test MSE:',lms_metrics_test[0])
```

```
Training MSE: 0.43919976805833433
Test MSE: 0.5212740055076005
```

Standardizing the features causes the coefficients to change but the model is effectively the same and has the same predictive performance as the previous model. The coefficients now reflect the change in `lpsa` if the feature is increased by one standard deviation. If all variables were increased by one standard deviation, we see that `lcavol` would give the largest change in `lpsa`. This makes it easier to compare coefficient values across features.

5 Ridge Regression

Ridge regression is a natural extension to linear regression which introduces an ℓ_2 penalty on the coefficients in a standard least squares problem.

The Ridge model is provided by the `linear_model` submodule. Note that the penalty parameter (referred to as λ in the lecture notes) is called `alpha` is sklearn, and, as discussed in lectures, this parameter crucially determines the amount of shrinkage towards zero and the weight of the ℓ_2 penalty.

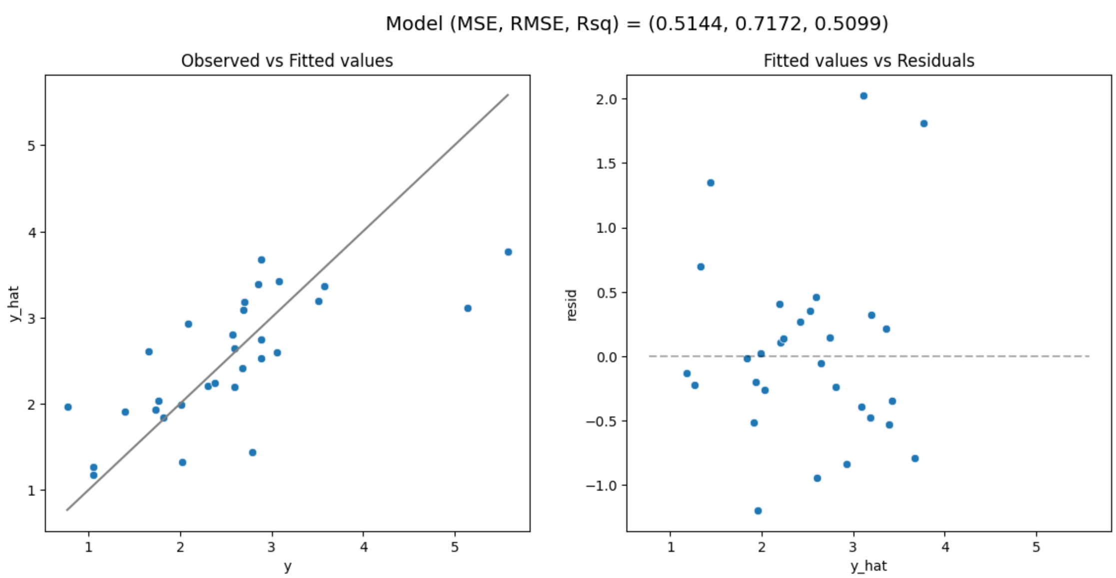
After defining the ridge regression model via, e.g. `Ridge(alpha = 1)`, the usual methods can be called, such as `.fit()` to fit the model and `.predict()` to make predictions.

As for the `LinearRegression()`, after fitting, the intercept and coefficients are stored separately in the attributes `.intercept_` and `.coef_`. In Ridge, this is helpful as it highlights how the penalty is only applied to the coefficient (i.e. we do not want to shrink the intercept).

Let's start by fitting a ridge regression model with $\alpha = 1$.

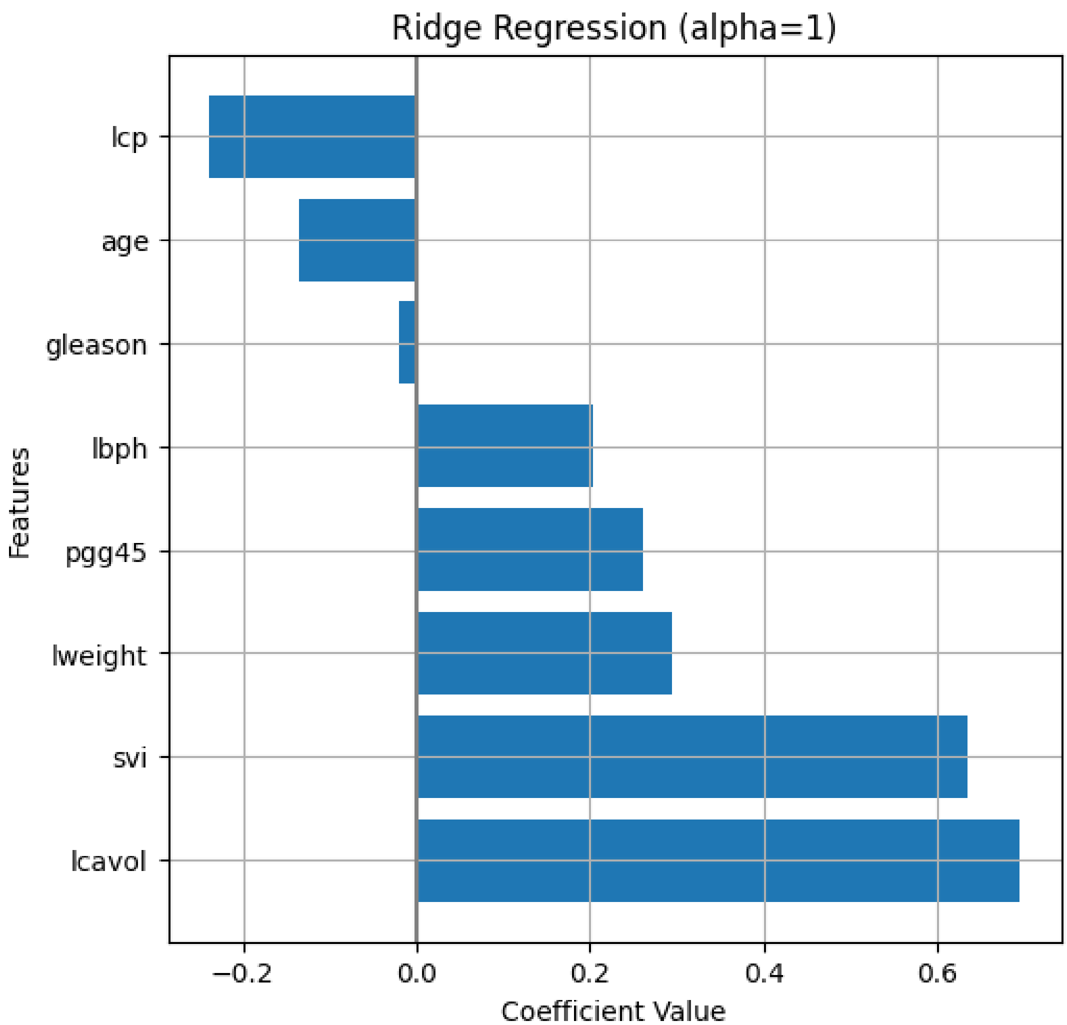
[20]:
```
from sklearn.linear_model import Ridge
```

[21]:
```
# Selected alpha value
alpha_val = 1

# Ridge pipeline
r = make_pipeline(
    preprocessor,
    Ridge(alpha = alpha_val)
).fit(X_train, y_train)

model_fit(r, X_test, y_test, plot = True)
```



[21]: (0.5143980978232647, 0.7172155169983878, 0.5099305592086348)

[22]:
```
w_dr_r = get_coefs(r, plot=True, figsize=(6,6), figtitle="Ridge Regression␣
↪(alpha="+str(alpha_val)+")", intercept=False)
```



5.0.1 Exercise 4 (CORE)

Adjust the value of `alpha` in the cell above and rerun it. Qualitatively, how does the model fit change as alpha changes? How does the MSE change?

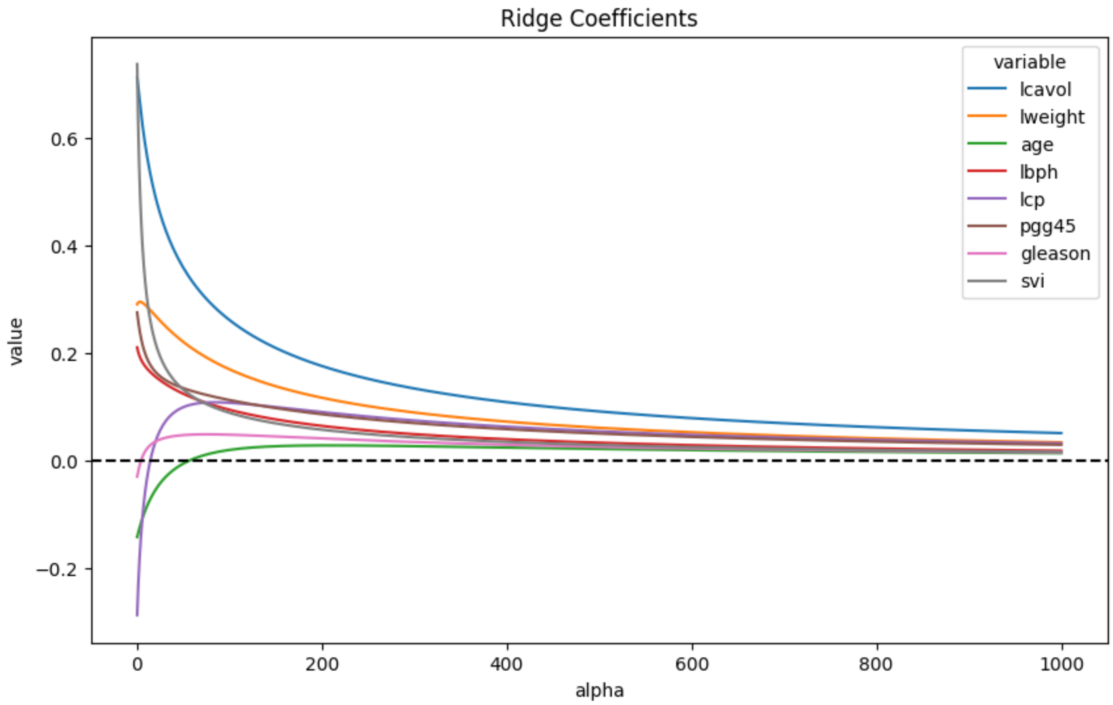
The model fit does not change much but MSE improves as alpha is increased until around $\alpha = 20$.

5.1 Solution path: Ridge coeffcients as a function of α

A useful way of examining the behavior of Ridge regression models is to plot the **solution path** of the coefficents $\mathbf{w}$ as a function of the penalty parameter α. Since Ridge regression is equivalent to linear regression when $\alpha = 0$, we can see that as we increase the value of α, we are shrinking all of the coefficients in $\mathbf{w}$ towards zero asymptotically α approaches infinity.

[23]:
```
# Grid of alpha values
alphas = np.logspace(-2, 3, num=200) # from 10^-2 to 10^3

ws = [] # Store coefficients
mses_train = [] # Store training mses
mses_test = [] # Store test mses

for a in alphas:
    m = make_pipeline(
        preprocessor,
        Ridge(alpha=a)
    ).fit(X_train, y_train)

    ws.append(m[-1].coef_)
    mses_train.append(mean_squared_error(y_train, m.predict(X_train)))
    mses_test.append(mean_squared_error(y_test, m.predict(X_test)))
```

[24]:
```
# Create a data frame for plotting
sol_path = pd.DataFrame(
    data = ws,
    columns = m[0].get_feature_names_out()
).assign(
    alpha = alphas,
).melt(
    id_vars = ('alpha')
)

# Plot solution path of the weights
plt.figure(figsize=(10,6))
ax = sns.lineplot(x='alpha', y='value', hue='variable', data=sol_path)
ax.axhline(y=0, color = "black", linestyle='dashed')
ax.set_title("Ridge Coefficients")
plt.show()
```



5.1.1 Exercise 5 (CORE)

Based on this plot, which variable(s) seem to be the most important for predicting `lpsa`?

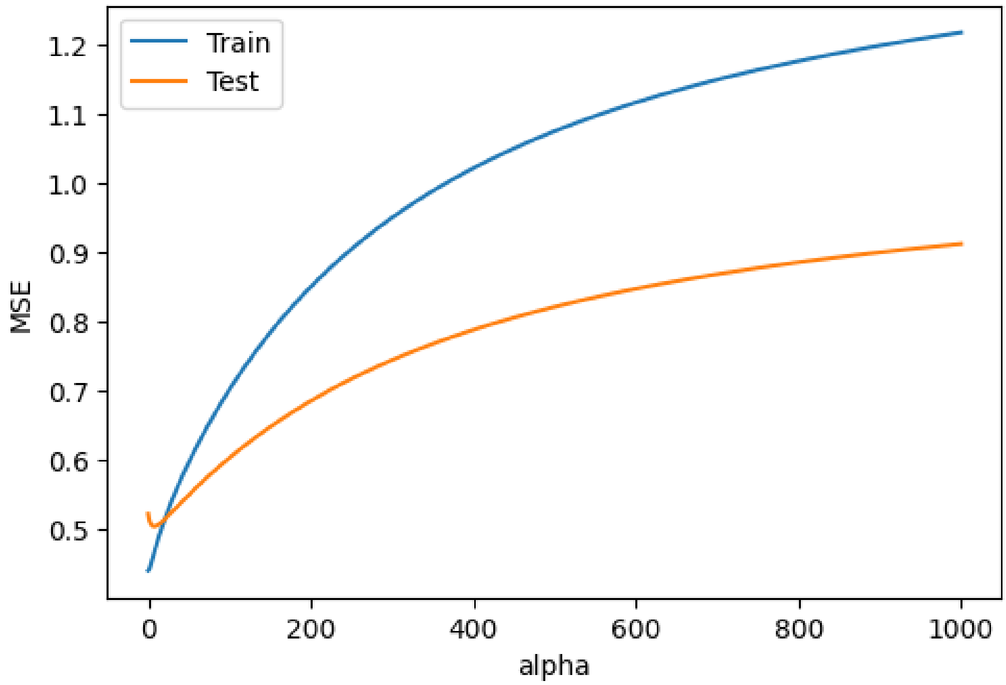
It appears that `lcavol` is still included in the model even for higher values of α, and is therefore likely to be the most important variable for this model.

5.1.2 Exercise 6 (CORE)

Run the code below to also plot both the training and test MSE as a function of α. What do you notice about the MSE as we increase α? Which value of α seems better regarding the changes on training and testing MSE values?

[25]:
```
# Path of MSE as function of alpha
mses_path = pd.DataFrame(
    {'alpha': alphas, 'Train': np.asarray(mses_train), 'Test': np.
 ↪asarray(mses_test)}).melt(
    id_vars = ('alpha')
)

# Plot MSE path
plt.figure(figsize=(6,4))
ax = sns.lineplot(x='alpha', y='value', hue='variable', data=mses_path)
ax.set_ylabel("MSE")
# ax.set_xlim(0,200) # Optional: to zoom in on lower alpha values
```

```
# To remove legend title
handles, labels = ax.get_legend_handles_labels()
ax.legend(handles=handles[0:], labels=labels[0:])
plt.show()
```



The training MSE increases as a function of α as expected, since the larger alpha imposes a stronger penalty (decreasing model complexity). The test MSE has more of U-shape, at first decreasing and then increasing when the penalty is too strong.

[26]:
```
alphas[np.argmin(np.asarray(mses_test))]
```

[26]: 7.3168071434272

5.2 Tuning the penalty parameter with cross-validation

We see that the value of α crucially determines the performance of the ridge regression model. While `RidgeRegression()` uses the default value of `alpha=1`, this **should never be used in practice**. Instead, this parameter can be **tuned using cross-validation**.

As with the polynomial models from last week, we can use `GridSearchCV` to employ k-fold cross validation to determine an optimal α. Remember, you can use the method `.get_params()` on your pipeline to list the parameters names to specify in `GridSearchCV`.

[27]:
```
# Grid of tuning parameters
alphas = np.linspace(0, 30, num=201)
```

```python
#Pipeline
m = make_pipeline(
        preprocessor,
        Ridge())
# To get the parameter name for grid search
# m.get_params()

# CV strategy
cv = KFold(5, shuffle=True, random_state=1234)

# Grid search
gs = GridSearchCV(m,
    param_grid={'ridge__alpha': alphas},
    cv=cv,
    scoring="neg_mean_squared_error")
gs.fit(X_train, y_train)
```

[27]:

```
GridSearchCV(cv=KFold(n_splits=5, random_state=1234, shuffle=True),
             estimator=Pipeline(steps=[('columntransformer',
ColumnTransformer(transformers=[('standardscaler',
StandardScaler(),
['lcavol',
'lweight',
                                                                          'age',
'lbph',
                                                                          'lcp',
'pgg45']),
('ordinalencoder',
OrdinalEncoder(categories=[[6,
                     7,
                     8,
                     9]]),
['gleason']),
('passthrough',
'passthrough',
['svi'])],
                                                         verbose_featu…
       20.25, 20.4 , 20.55, 20.7 , 20.85, 21.  , 21.15, 21.3 , 21.45,
       21.6 , 21.75, 21.9 , 22.05, 22.2 , 22.35, 22.5 , 22.65, 22.8 ,
       22.95, 23.1 , 23.25, 23.4 , 23.55, 23.7 , 23.85, 24.  , 24.15,
       24.3 , 24.45, 24.6 , 24.75, 24.9 , 25.05, 25.2 , 25.35, 25.5 ,
       25.65, 25.8 , 25.95, 26.1 , 26.25, 26.4 , 26.55, 26.7 , 26.85,
       27.  , 27.15, 27.3 , 27.45, 27.6 , 27.75, 27.9 , 28.05, 28.2 ,
       28.35, 28.5 , 28.65, 28.8 , 28.95, 29.1 , 29.25, 29.4 , 29.55,
       29.7 , 29.85, 30.  ])},
             scoring='neg_mean_squared_error')
```

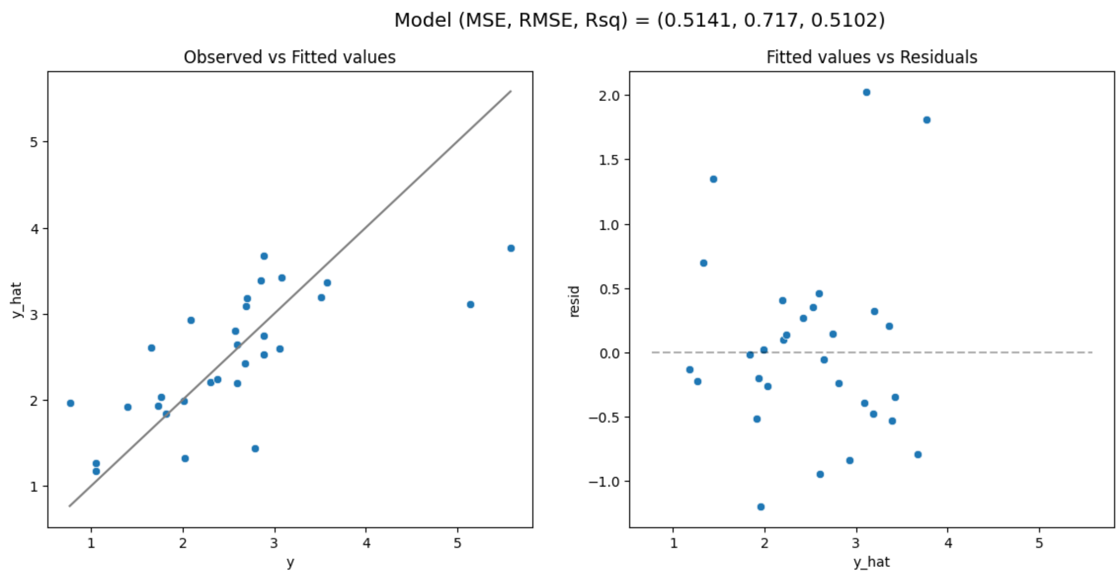
Note that we are passing `sklearn.model_selection.KFold(5, shuffle=True, random_state=1234)` to the `cv` argument rather than leaving it to its default. This is because, while not obvious, the prostate data is structured (sorted by `lpsa` value) and this way we are able to ensure that the folds are properly shuffled. Failing to do this causes *very* unreliable results from the cross validation process.

Once fit, we can examine the results to determine what value of α was chosen as well as examine the results of cross validation.

[28]:
```
print(gs.best_params_)
print(-gs.best_score_)
```

```
{'ridge__alpha': 1.05}
0.7113777058489017
```

[29]:
```
model_fit(gs.best_estimator_, X_test, y_test, plot=True)
```



[29]: (0.5141325717194642, 0.7170303840978179, 0.510183527152637)

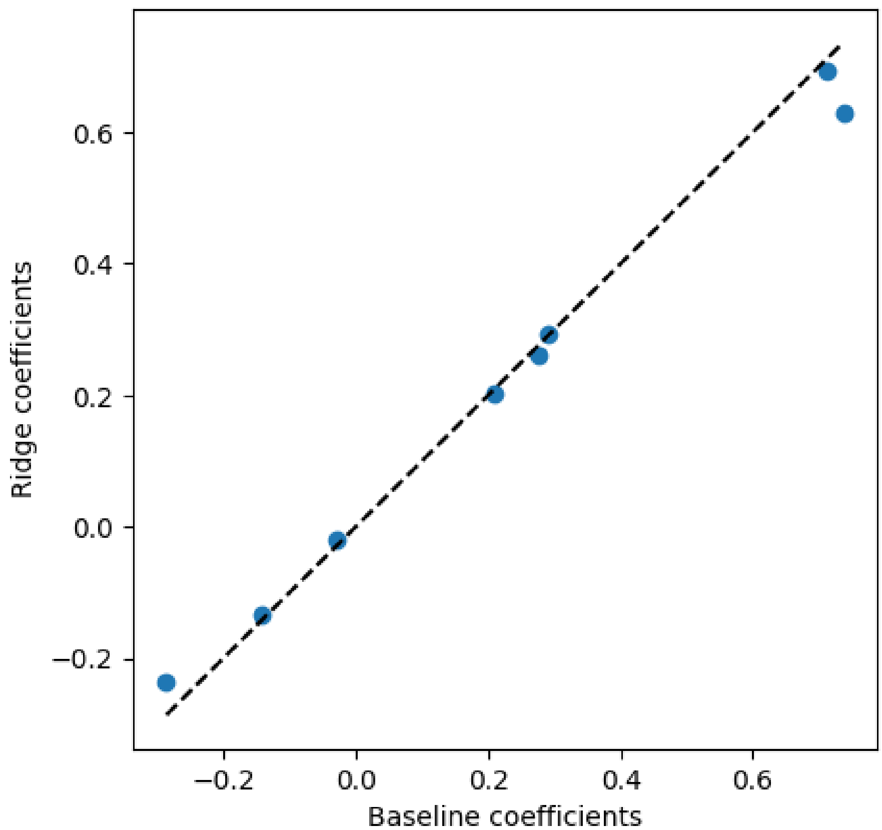
5.2.1 Exercise 7 (CORE)

- How does this model compare to the performance of our baseline model? Is it better or worse?
- How do the model coefficients for this model compare to the baseline model? To answer this, create a scatter plot of the coefficients for the baseline model against the coefficients for the ridge model. Are they always higher or lower? Now, use `np.linalg.norm` to compute the ℓ_2 norm of the coeffcients for both models and comment on the results.

The model performs better than our baseline model (linear regression), as it has a lower RMSE and higher R^2.

When comparing the coefficients in the plot below, we find that some coefficients are smaller and some are larger in the ridge model compared to the baseline model. Generally, this may be the case for the ridge coefficients. However, overall, the ℓ_2 norm of the ridge regression coefficients will be smaller (as shown below for this example).

[30]:
```
# Plot of linear regression vs ridge coefficients
fig, ax = plt.subplots(figsize=(5,5),nrows=1,ncols=1)
ax.scatter(lm_s[1].coef_, gs.best_estimator_[1].coef_)
x1 = np.min(lm_s[1].coef_)
x2 = np.max(lm_s[1].coef_)
ax.plot([x1,x2],[x1,x2],color='k',linestyle="dashed")
ax.set_xlabel('Baseline coefficients')
ax.set_ylabel('Ridge coefficients')
plt.show()
```



[31]:
```
print('l2 norm of baseline coeffcients',np.linalg.norm(lm_s[1].coef_))
print('l2 norm of ridge coeffcients',np.linalg.norm(gs.best_estimator_[1].
 ↪coef_))
```

```
l2 norm of baseline coeffcients 1.1647449934510492
l2 norm of ridge coeffcients 1.072261915103183
```

As we saw last week, it is also recommend to plot the CV scores. Although the grid search may report a best value for the parameter corresponding to the maximum CV score (e.g. min CV MSE), if the curve is relatively flat around the minimum, we prefer the simpler model.

Recall from last week that we can access the cross-validated scores (along with other results for each split) in the attribute `cv_results_`.

[32]:
```
cv_results = pd.DataFrame(gs.cv_results_)
cv_results.head()
```

[32]:
```
   mean_fit_time  std_fit_time  mean_score_time  std_score_time  \
0       0.004066      0.000991         0.002020        0.000129
1       0.003468      0.000148         0.001970        0.000118
2       0.003429      0.000215         0.001940        0.000133
3       0.003964      0.000782         0.002099        0.000253
4       0.003403      0.000082         0.001910        0.000059

   param_ridge__alpha                                 params  \
0                0.00                {'ridge__alpha': 0.0}
1                0.15               {'ridge__alpha': 0.15}
2                0.30                {'ridge__alpha': 0.3}
3                0.45  {'ridge__alpha': 0.44999999999999996}
4                0.60                {'ridge__alpha': 0.6}

   split0_test_score  split1_test_score  split2_test_score  split3_test_score  \
0          -0.933523          -0.684212          -0.732690          -0.424796
1          -0.936583          -0.689944          -0.730721          -0.419826
2          -0.939497          -0.695286          -0.729046          -0.415606
3          -0.942272          -0.700271          -0.727639          -0.412013
4          -0.944917          -0.704931          -0.726475          -0.408949

   split4_test_score  mean_test_score  std_test_score  rank_test_score
0          -0.789296        -0.712903        0.166571               20
1          -0.785028        -0.712420        0.168506               16
2          -0.780848        -0.712057        0.170243               13
3          -0.776750        -0.711789        0.171809               10
4          -0.772727        -0.711600        0.173226                8
```

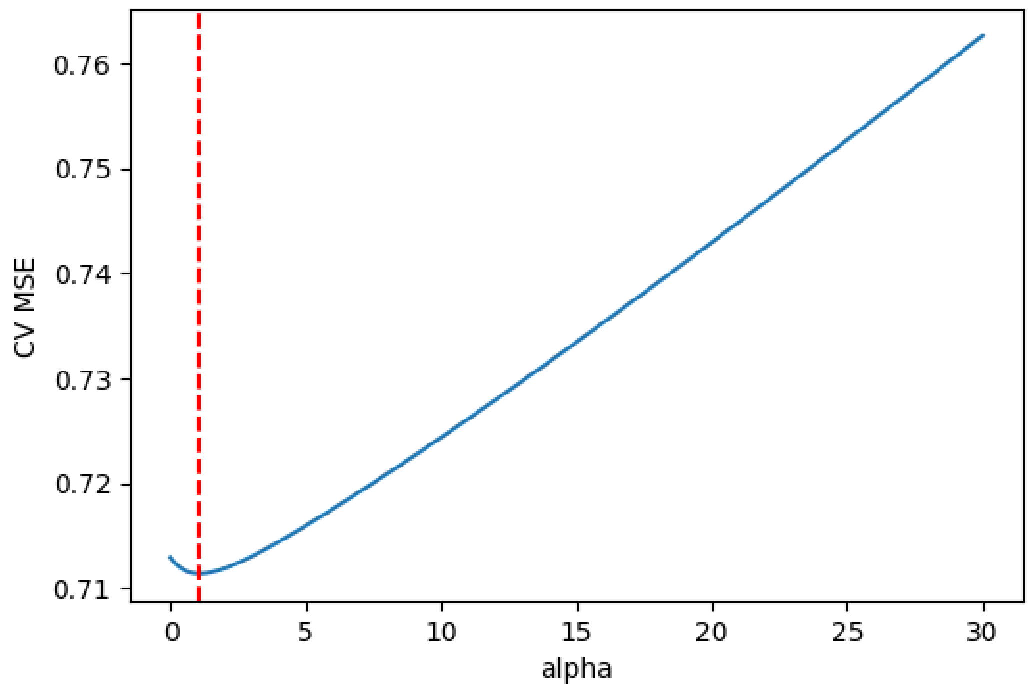
In particular, let's examining the `mean_test_score` and the `split#_test_score` keys since these are used to determine the optimal α.

In the code below we extract these data into a data frame by selecting our columns of interest along with the α values used (and transform negative MSE values into positive values).

[33]:
```
# Extract only mean and split scores
cv_mse = pd.DataFrame(
    data = gs.cv_results_
).filter(
    # Extract the split#_test_score and mean_test_score columns
```

```
    regex = '(split[0-9]+|mean)_test_score'
).assign(
    # Add the alphas as a column
    alpha = alphas
)

cv_mse.update(
    # Convert negative mses to positive
    -1 * cv_mse.filter(regex = '_test_score')
)
```

[34]:
```
# Plot CV MSE
plt.figure(figsize=(6,4))
ax = sns.lineplot(x='alpha', y='mean_test_score', data=cv_mse)
ax.set_ylabel('CV MSE')
ax.axvline(x=gs.best_params_['ridge__alpha'], color='red', linestyle='dashed',␣
 ↪label='Best alpha')
plt.show()
```

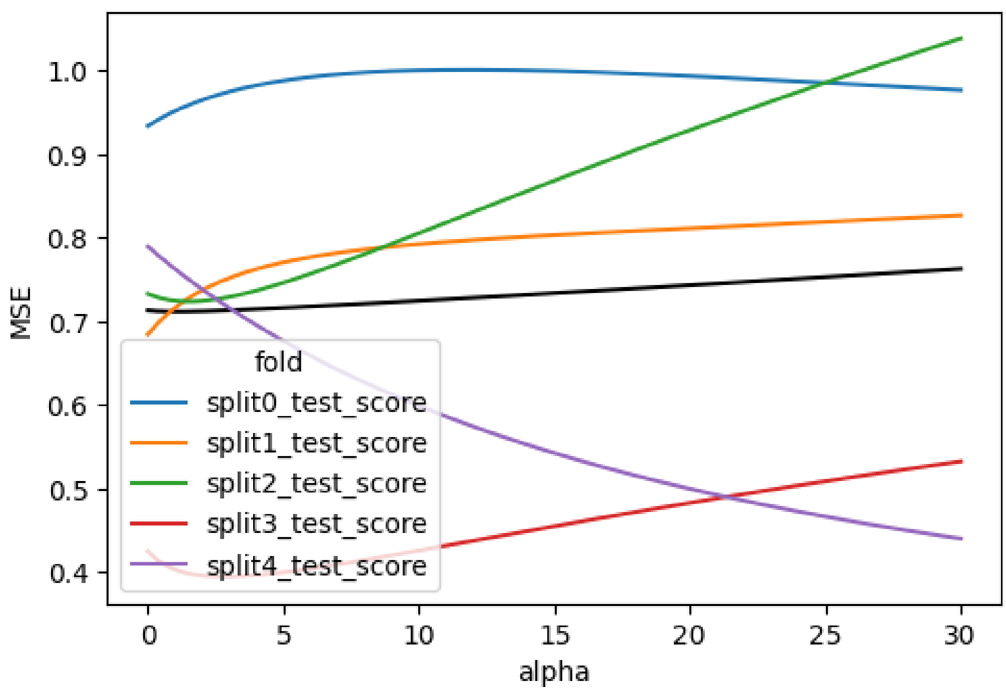


This plot shows that the value of $\alpha = 1.05$ corresponds to the minimum of this curve. However, this plot gives us an overly confident view of this particular value of α. Specifically, if instead of just plotting the mean MSE across all of the validation sets, we also examine the MSE for each fold individually and the corresponding optimal value of α, we see that there is a lot of noise in the MSE and we should take the value $\alpha = 1.05$ with a grain of salt.

5.2.2 Exercise 8 (CORE)

Run the code below to plot the MSE for each validation set in the 5-fold cross validation. Why do you think that our cross validation results are unstable?

[35]:
```
# Reshape the data frame for plotting
d = cv_mse.melt(
    id_vars=('alpha','mean_test_score'),
    var_name='fold',
    value_name='MSE'
)

# Plot the validation scores across folds
plt.figure(figsize=(6,4))
sns.lineplot(x='alpha', y='MSE', color='black', errorbar=None, data = d)  #␣
 ↪Plot the mean MSE in black.
sns.lineplot(x='alpha', y='MSE', hue='fold', data = d) # Plot the curves for␣
 ↪each fold in different colors
plt.show()
```

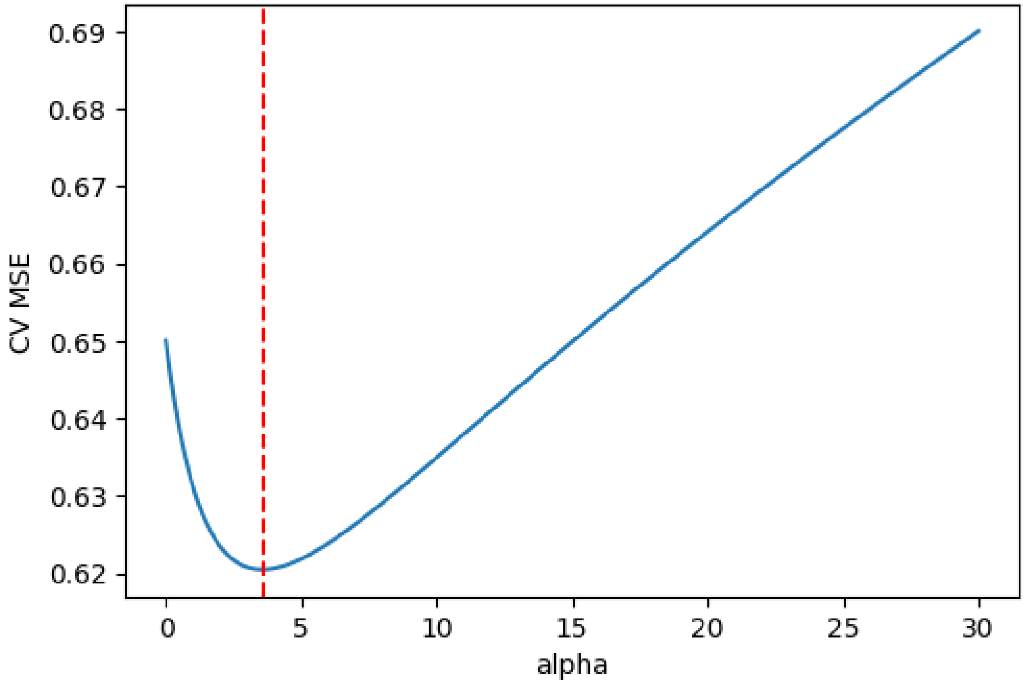


The size of training data is $N = 67$, which is relatively small. Thus, when we perform five fold cross-validation, for each fold, the number of points in the training and validation sets are small, with only 54 (or 53) and 13 (or 14) observations in each, respectively. This produces both unstable estimates of the coefficients when training for each fold and unstable MSE when evaluating on the validation set.

5.2.3 Exercise 9 (CORE)

Lastly, try changing the random seed in the cross-validation scheme. Plot the CV MSE with the optimal value of alpha marked with a vertical dashed line. How do the results change? Are different values of `alpha` suggested? Comment on your preffered value of `alpha`.

[36]:
```python
# CV strategy
cv = KFold(5, shuffle=True, random_state=0)

# Grid search
gs = GridSearchCV(m,
    param_grid={'ridge__alpha': alphas},
    cv=cv,
    scoring="neg_mean_squared_error")
gs.fit(X_train, y_train)

# Extract only mean and split scores
cv_mse = pd.DataFrame(data = gs.cv_results_).filter(
    regex = 'mean_test_score' # Extract the mean_test_score column
).assign(
    alpha = alphas # add column of alphas
)

cv_mse.update(
    # Convert negative mses to positive
    -1 * cv_mse.filter(regex = 'mean_test_score')
)

# Plot CV MSE
plt.figure(figsize=(6,4))
ax = sns.lineplot(x='alpha', y='mean_test_score', data=cv_mse)
ax.set_ylabel('CV MSE')
ax.axvline(x=gs.best_params_['ridge__alpha'], color='red', linestyle='dashed',␣
 ↪label='Best alpha')
plt.show()
```



[37]:
```
print(gs.best_params_)
```

```
{'ridge__alpha': 3.5999999999999996}
```

The optimal value of `alpha`=3.6 is now larger. Combining this with the variability across folds, suggests that picking a larger value of `alpha` corresponding to a simpler model may be preferable.

Note: Due to the importance of tuning the value of α in ridge regression, sklearn provides a function called `RidgeCV` which combines `Ridge` with `GridSearchCV`. However, we will avoid using this function for two reasons:

- it does not allow us to account for additional steps in our pipeline such as standardization when carrying out cross validation, resulting in *data leakage*
- it only allows storing all results of the cross-validation in the attribute `.cv_results_` in the case of the default leave-one-out cross validation, with option `store_cv_results=True`. So, if you want to access all results and use a cross-validation strategy other than leave-one-out, you will need to use `GridSearchCV`.

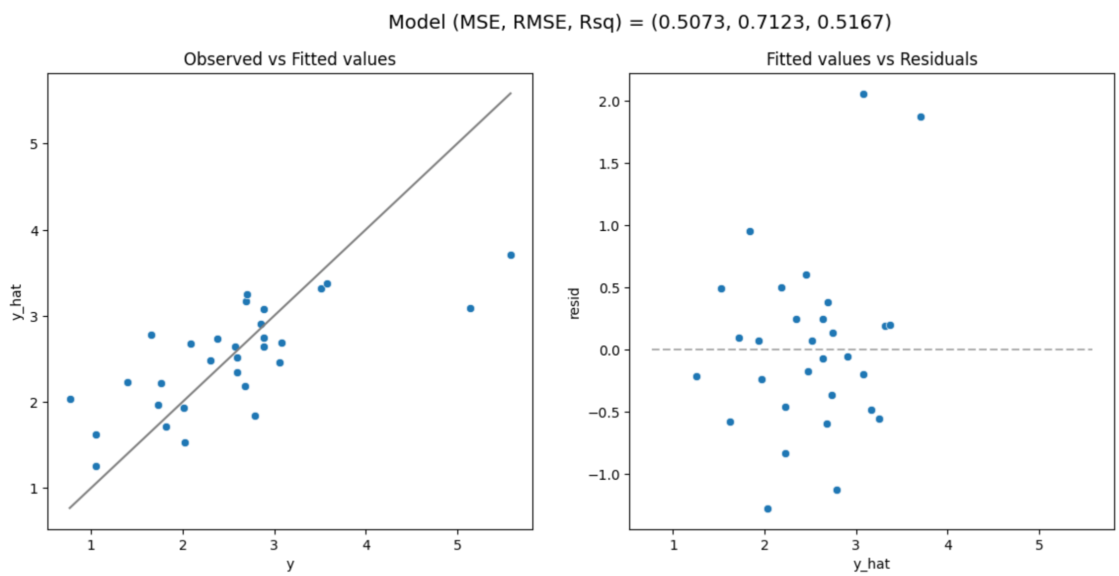
6 Lasso Regression

We saw that ridge regression with a wise choice of α can outperform our baseline linear regression. We can now investigate if lasso can yield a more accurate or interpretable solution. Recall that lasso uses an ℓ_1 penalty on the coefficients, as opposed to the ℓ_2 penalty of ridge.

The `Lasso` model is also provided by the `linear_model` submodule and similarly requires the choice of the tuning parameter `alpha` to determine the weight of the ℓ_1 penalty.

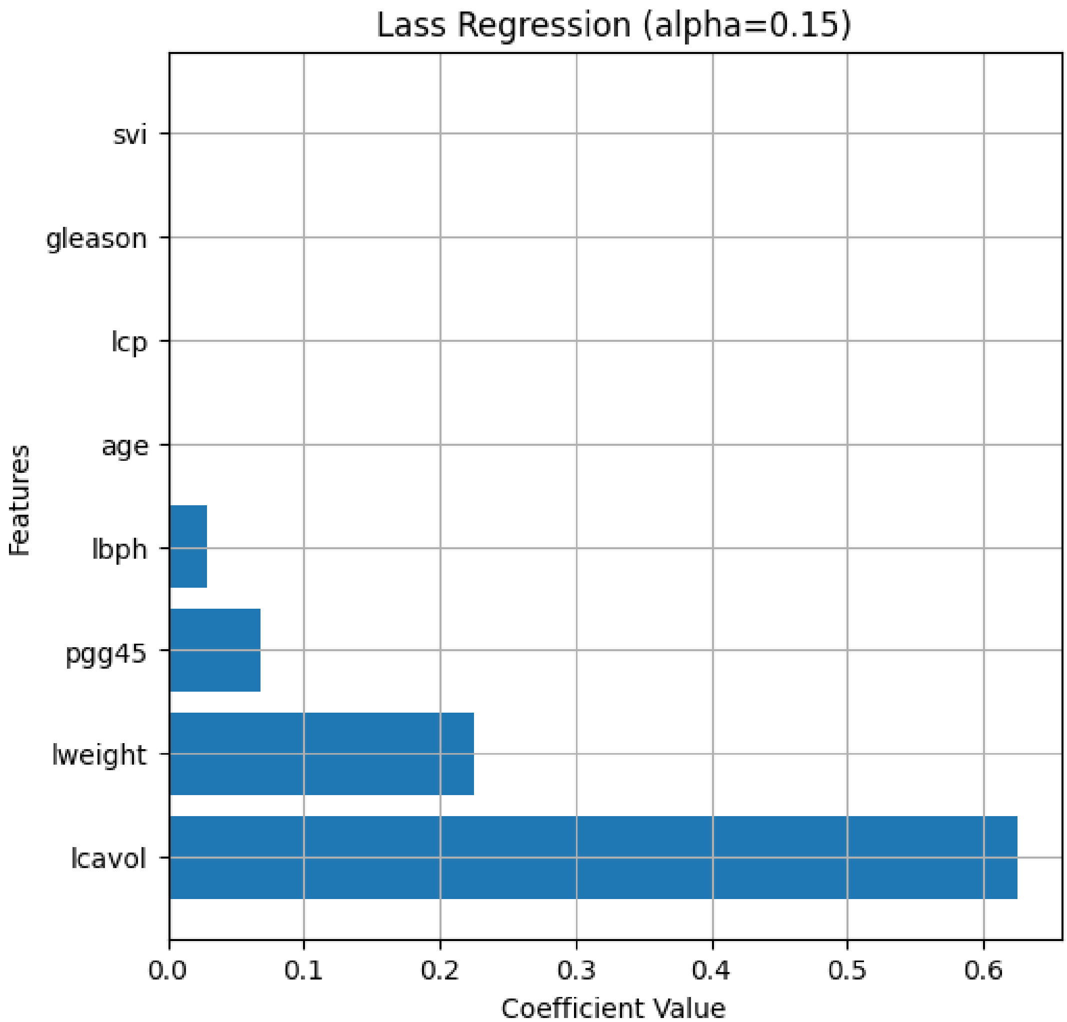
Try running the code below with different values of α to see how it effects sparsity in the coefficients and model performance.

[38]:
```
from sklearn.linear_model import Lasso

# Selected alpha value
alpha_val = 0.15

# Lasso pipeline
l = make_pipeline(
    preprocessor,
    Lasso(alpha = alpha_val)
).fit(X_train, y_train)

model_fit(l, X_test, y_test, plot = True)
```



[38]: (0.5073090826556591, 0.7122563321274575, 0.5166842966614968)

[39]:
```
w_dr_r = get_coefs(l, plot=True, figsize=(6,6), figtitle="Lass Regression␣
↪(alpha="+str(alpha_val)+")", intercept=False)
```



6.0.1 Exercise 10 (CORE)

a) Plot the solution path of the coefficients as a function of α.

b) How does this differ between the solution path for Ridge for large α? for small α?

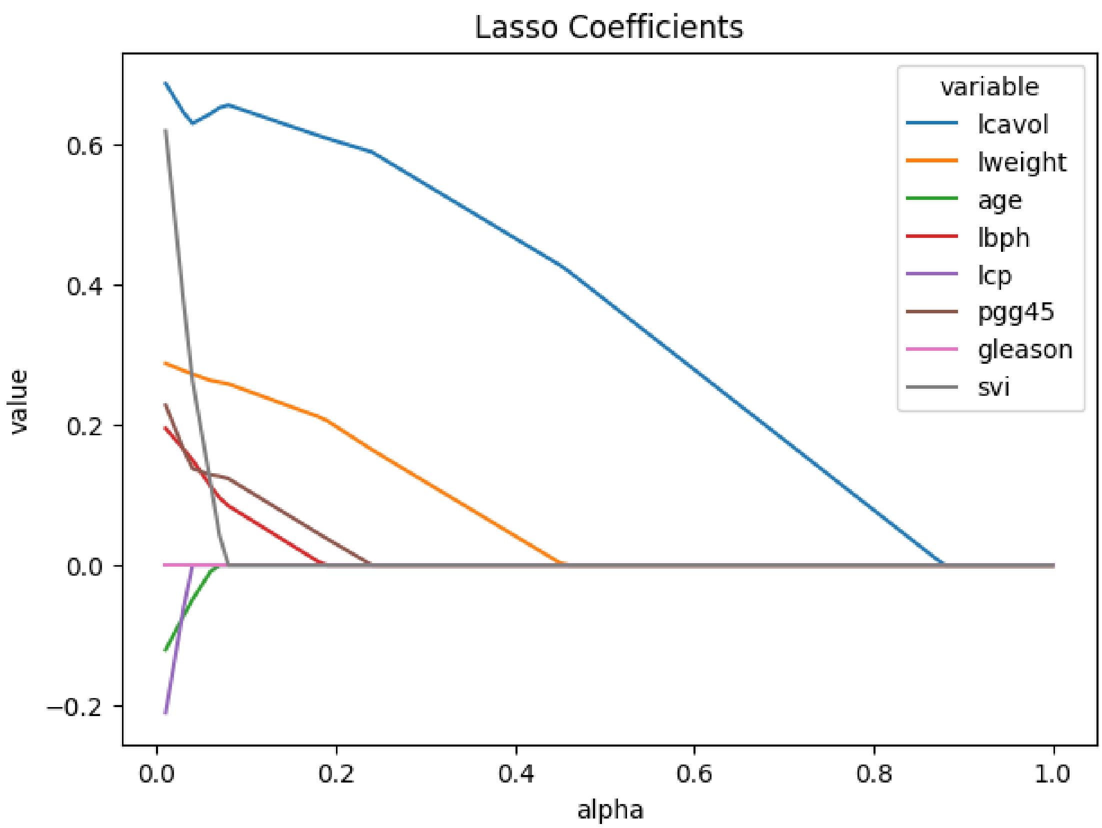
c) Which variable seems to be the most important for predicting `lpsa`?

Note that $\alpha = 0$ causes a warning due to the fitting method (coordinate descent) not converging well without regularization (the ℓ_1 penalty here). So, the grid of α values needs to start at some small positive constant.

[40]:
```
# Part a: Compute the solution path
alphas = np.linspace(0.01, 1, num=100)

ws = [] # Store coefficients
mses_train = [] # Store training mses
```

```
mses_test = [] # Store test mses

for a in alphas:
    m = make_pipeline(
        preprocessor,
        Lasso(alpha=a)
    ).fit(X_train, y_train)

    ws.append(m[1].coef_)
    mses_train.append(mean_squared_error(y_train, m.predict(X_train)))
    mses_test.append(mean_squared_error(y_test, m.predict(X_test)))
```

[41]:
```
# Part a: Plot the solution path
# Create a data frame for plotting
sol_path = pd.DataFrame(
    data = ws,
    columns = m[0].get_feature_names_out()
).assign(
    alpha = alphas,
).melt(
    id_vars = ('alpha')
)

# Plot the solution path of the weights
plt.figure(figsize=[7,5])
ax = sns.lineplot(x='alpha', y='value', hue='variable', data=sol_path)
ax.set_title("Lasso Coefficients")
plt.show()
```



b) The values of the coefficients are shrunk to exactly zero as α increases. For small α, ridge and lasso are both similar to the linear regression coefficients.

c) The variable `lcavol` seems to be the most important, as for values of α approximately between 0.5 and 0.9, it is the only variable included (i.e. it is the last variable whose coefficient is shrunk towards zero).

6.1 Tuning the Lasso penalty parameter

Again, we can use the `GridSearchCV` function to tune our Lasso model and optimize the α hyperparameter. You could also use LassoCV, which combines `Lasso` and `GridSearchCV` but we will focus on the former to avoid *data leakage*.

6.1.1 Exercise 11 (CORE)

a) Use `GridSearchCV` to find the optimal value of α.

b) Plot the CV MSE and MSE for each fold. Comment on the stability and uncertainty of α across the different folds. Try changing the value of the `random_state` in the CV strategy - do the results change? are they more or less stable compared with ridge?

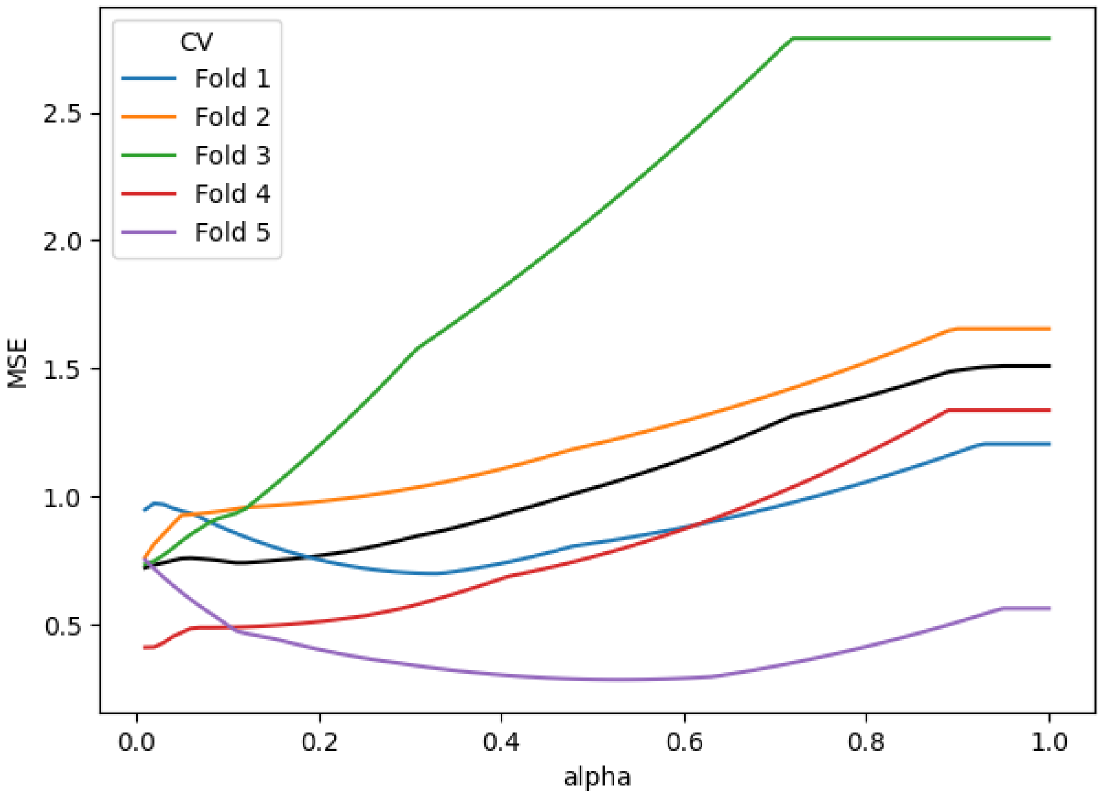
c) Which variables are included with this optimal value of α?

[42]:
```python
# Grid of tuning parameters
alphas = np.linspace(0.01, 1, num=100)

# Pipeline
m = make_pipeline(
        preprocessor,
        Lasso())

# CV strategy
cv = KFold(5, shuffle=True, random_state=1234)

# Grid search
gs_lasso = GridSearchCV(m,
                        param_grid={'lasso__alpha': alphas},
                        cv = cv,
                        scoring="neg_mean_squared_error")
gs_lasso.fit(X_train, y_train)

print( "alpha:", gs_lasso.best_params_)
```

```
alpha: {'lasso__alpha': 0.01}
```

[43]:
```python
# Extract only split scores
cv_mse = pd.DataFrame(
    data = gs_lasso.cv_results_
).filter(
    # Extract the split#_test_score and mean_test_score columns
    regex = '(split[0-9]+)_test_score'
)

# Convert negative mses to positive
cv_mse.update(-1 * cv_mse.filter(regex = '_test_score'))

# Rename columns
fold_names = ['Fold '+str(i+1) for i in range(5)]
cv_mse.columns = fold_names

# Add the alphas as a column
cv_mse['alpha'] = alphas

# Reshape data frame for plotting
d = cv_mse.melt(
    id_vars=('alpha'),
    var_name='CV',
    value_name='MSE'
)

# Plot the MSE path
```

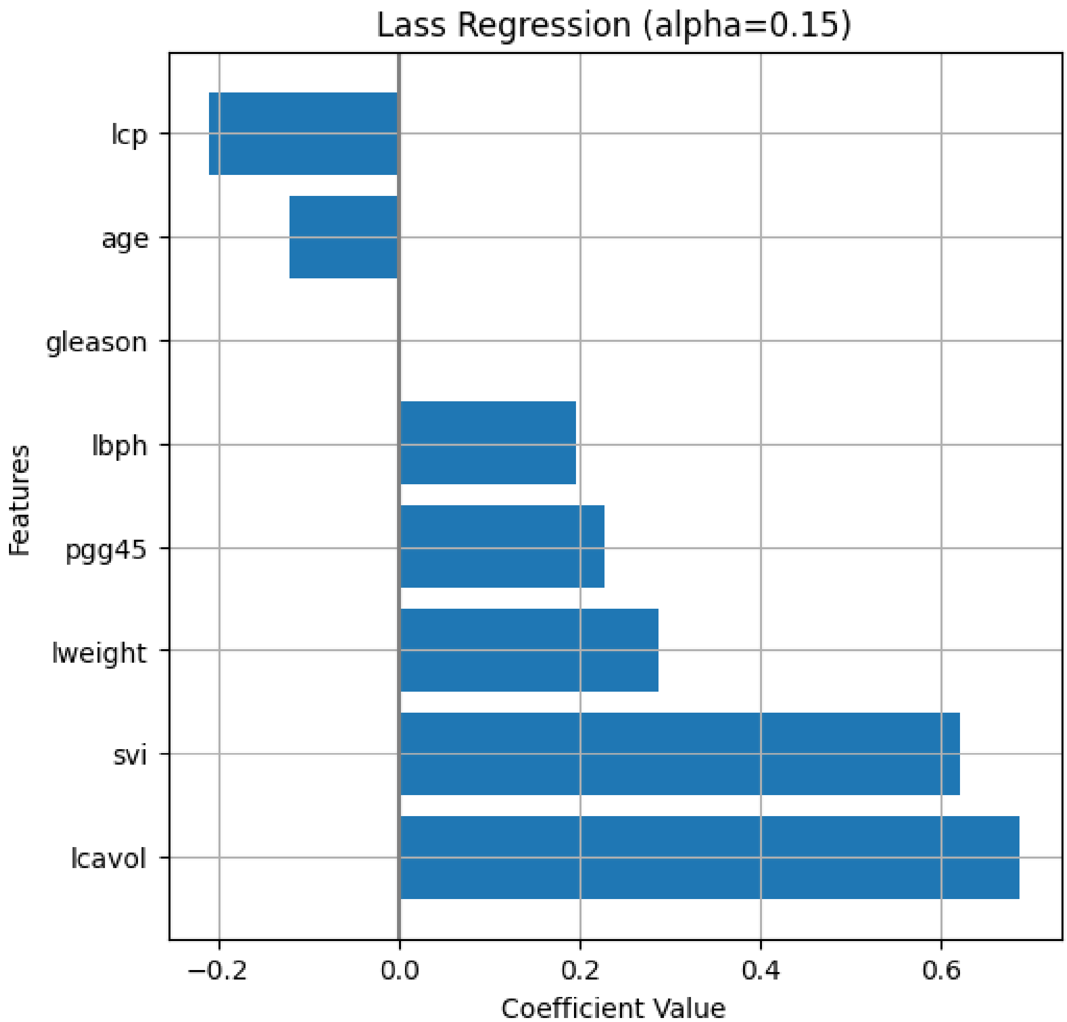
```
plt.figure(figsize=[7,5])
sns.lineplot(x='alpha', y='MSE', color='black', errorbar=None, data = d)  #␣
↪Plot the mean MSE in black.
sns.lineplot(x='alpha', y='MSE', hue='CV', data = d) # Plot the curves for each␣
↪fold in different colors
plt.show()
```



In this case, the optimal value of α is the minimum value of the grid. Three of the five folds suggest a similar value of α, and it seems to be slightly more stable than in the case of ridge. Here, we don't observe the anticipated U-shape in the CV MSE, which may be a bit worrying and suggests that we may need to adjust our grid to include smaller values and/or that the baseline linear regression model (i.e. $\alpha = 0$) should be chosen.

If we change our random seed, the results may change. In some cases they don't, but in others e.g. `random_state=0` a slightly larger value of `alpha` is selected, resulting in a sparser set of coefficients. Overall though, the results seems to be slightly more stable than ridge.

[44]:
```
# Part c
w_df_l = get_coefs(gs_lasso.best_estimator_, plot=True, figsize=(6,6),␣
↪figtitle="Lass Regression (alpha="+str(alpha_val)+")", intercept=False)
```



All variables except `gleason` are included in the model.

6.1.2 Exercise 12 (CORE)

Run the following code to compute the CV MSE for the linear model and compare with the CV MSE of the lasso model to suggest an optimal value of α.

[45]:
```
# Lasso doesn't allow for alpha=0, so compute CV MSE for linear regression␣
↪model to compare with Lasso
gs_l = GridSearchCV(
    make_pipeline(
        preprocessor,
        LinearRegression()
    ),
    param_grid = {},
```

```
    cv=KFold(5, shuffle=True, random_state=1234),
    scoring="neg_mean_squared_error"
).fit(X_train, y_train)
```

[46]:
```
print('CV MSE for baseline linear model', round(gs_l.best_score_ * -1,4))
print('CV MSE for lasso model', round(gs_lasso.best_score_ * -1,4))
```

```
CV MSE for baseline linear model 0.7129
CV MSE for lasso model 0.7226
```

The CV MSE is lower for the baseline linear regression, suggesting that the optimal value is $\alpha = 0$.

7 ElasticNet Regression

Lastly, we can use elastic net regression, which is hybrid between lasso and ridge, including both an ℓ_1 and ℓ_2 penalty. The `ElasticNet` model is again provided by the `linear_model` submodule and minimizes the objective:

$$\frac{1}{2N}||\mathbf{y}-\mathbf{X}\mathbf{w}||_2^2 + \alpha\rho||\mathbf{w}||_1 + 0.5\alpha(1-\rho)||\mathbf{w}||_2^2.$$

In this parameterization, ρ determines relative strength of the ℓ_1 penalty compared to the ℓ_2 and is referred to as `l1_ratio` in `ElasticNet`. Thus, we can also fit ridge and lasso regression models with `ElasticNet` through appropriate choice of `l1_ratio`: - ridge corresponds to `l1_ratio=0` - lasso corresponds to `l1_ratio=1`

The parameter α is referred to as `alpha` in `ElasticNet` and controls the overall penalty relative the residual sum of squares.

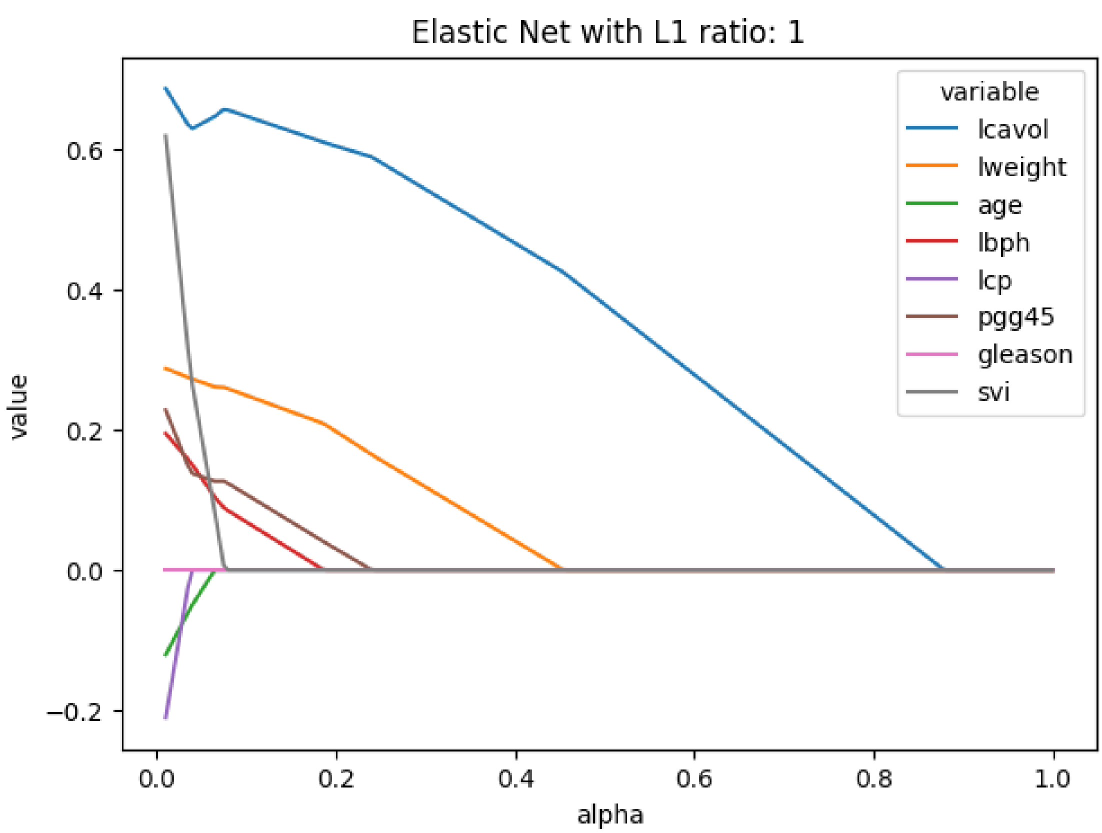
The general `ElasticNet` requires tuning of both `alpha` and `l1_ratio`.

7.0.1 Exercise 13 (CORE)

The following code plots the solution path for a specific value of `l1_ratio`. Try changing the value of `l1_ratio` (you may also want to change the maximal value of `alpha` for better visualization). How do the solution paths change?

[47]:
```
from sklearn.linear_model import ElasticNet

# Grid of alpha values
alphas = np.linspace(0.01, 1, num=200)
# L1 ratio
l = 1
ws = [] # Store coefficients
mses_train = [] # Store training mses
mses_test = [] # Store test mses

for a in alphas:
    m = make_pipeline(
        preprocessor,
```

```
        ElasticNet(alpha=a, l1_ratio=l)
    ).fit(X_train, y_train)

    ws.append(m[-1].coef_)
    mses_train.append(mean_squared_error(y_train, m.predict(X_train)))
    mses_test.append(mean_squared_error(y_test, m.predict(X_test)))

# Create a data frame with the solution path
sol_path = pd.DataFrame(
    data = ws,
    columns = m[0].get_feature_names_out()
).assign(
    alpha = alphas,
).melt(
    id_vars = ('alpha')
)

# Plot the solution path of the weights
plt.figure(figsize=[7,5])
ax = sns.lineplot(x='alpha', y='value', hue='variable', data=sol_path)
ax.set_title(f'Elastic Net with L1 ratio: {l}')
plt.show()
```



The solution paths resemble ridge and lasso for small and large values of `l1_ratio` respectively.

7.1 Tuning with Grid Search CV

Again, we can use `GridSearchCV` (or `ElasticNetCV`) to tune the parameters. In the following code, we use `GridSearchCV` to tune both `alpha` and `l1_ratio`.

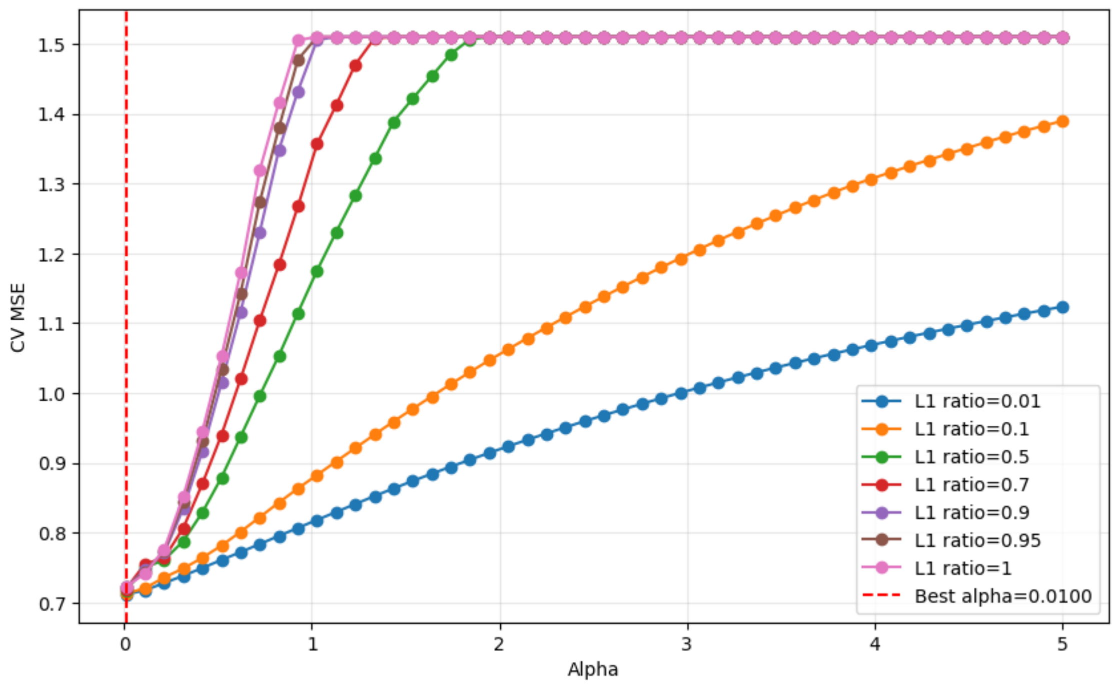
[48]:
```
# Grid of tuning parameters
alphas = np.linspace(0.01, 5, num=50)
l1r = [0.01, .1, .5, .7, .9, .95, 1]

# CV strategy
cv = KFold(5, shuffle=True, random_state=1234)

# Pipeline
m = make_pipeline(
        preprocessor,
        ElasticNet())

# Grid search
gs_enet = GridSearchCV(m,
                        param_grid={'elasticnet__alpha': alphas,␣
  ↪'elasticnet__l1_ratio': l1r},
                        cv = cv,
                        scoring="neg_mean_squared_error",
                        return_train_score=True)
gs_enet.fit(X_train, y_train)

gs_enet.best_params_
```

[48]:
```
{'elasticnet__alpha': 0.01, 'elasticnet__l1_ratio': 0.01}
```

Grid search returns the best value, but to better support these choices, let's plot the cv scores as a function of `alpha` for different values of `l1_ratio`.

[49]:
```
# Extract mean CV scores for each l1_ratio
cv_results_df = pd.DataFrame(gs_enet.cv_results_)
cv_results_df['alpha'] = cv_results_df['param_elasticnet__alpha']
cv_results_df['l1_ratio'] = cv_results_df['param_elasticnet__l1_ratio']

# Convert negative MSE to positive
cv_results_df['mean_cv_mse'] = -cv_results_df['mean_test_score']

# Plot
plt.figure(figsize=(10, 6))
for l1_val in l1r:
```

```
    subset = cv_results_df[cv_results_df['l1_ratio'] == l1_val]
    plt.plot(subset['alpha'], subset['mean_cv_mse'], marker='o', label=f'L1␣
↪ratio={l1_val}')

plt.axvline(x=gs_enet.best_params_['elasticnet__alpha'], color='red',␣
↪linestyle='dashed',
            label=f'Best alpha={gs_enet.best_params_["elasticnet__alpha"]:.4f}')
plt.xlabel('Alpha')
plt.ylabel('CV MSE')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



7.1.1 Exercise 14 (CORE)

Comment on the optimal values of `alpha` and `l1_ratio` for ElasticNet based on the plot above. How does the CV MSE of tuned Elastic Net compare to our basineline, ridge, and lasso models? How does the performance of the models compare on the test data?

The optimal value of the `l1_ratio` is the smallest value in the grid, suggesting that a smaller value may be needed or that ridge with `l1_ratio=0` may be the optimal value. In addition, the CV MSE curves tend to be smaller across `alpha` values when `l1_ratio` is lowered, furthering supporting a small value of `l1_ratio`. When we compare with the CV MSE of the ridge model, it indeed has a smaller CV MSE.

[50]:
```
print('CV MSE for elasticnet model', round(-gs_enet.best_score_,4))
print('CV MSE for ridge model',round(-gs.best_score_,4))
print('CV MSE for lasso model',round(-gs_lasso.best_score_,4))
print('CV MSE for baseline linear model',round(-gs_l.best_score_,4))
```

```
CV MSE for elasticnet model 0.7117
CV MSE for ridge model 0.6204
CV MSE for lasso model 0.7226
CV MSE for baseline linear model 0.7129
```

[51]:
```
print('Baseline linear model (MSE, RMSE, R2) =', model_fit(lm_s, X_test,␣
↪y_test, plot=False))
print('Ridge model (MSE, RMSE, R2) =', model_fit(gs, X_test, y_test, plot =␣
↪False))
print('Lasso model (MSE, RMSE, R2) =', model_fit(gs_lasso, X_test, y_test, plot␣
↪= False))
print('ElasticNet model (MSE, RMSE, R2) =', model_fit(gs_enet, X_test, y_test,␣
↪plot = False))
```

```
Baseline linear model (MSE, RMSE, R2) = (0.5212740055076005, 0.7219930785731955,
0.5033798502381805)
Ridge model (MSE, RMSE, R2) = (0.5060348662077456, 0.7113612768542757,
0.5178982485495747)
Lasso model (MSE, RMSE, R2) = (0.4996338263745379, 0.7068478099099819,
0.523996548727623)
ElasticNet model (MSE, RMSE, R2) = (0.5160834313205354, 0.7183894704967044,
0.5083249342110903)
```

The Ridge model performs slightly better on the test data, compared to the baseline and elastic net model, and similar to the Lasso model.

8 Competing the Worksheet

At this point you have hopefully been able to complete all the CORE exercises and attempted the EXTRA ones. Now is a good time to check the reproducibility of this document by restarting the notebook's kernel and rerunning all cells in order.

Before generating the PDF, please **change 'Student 1' and 'Student 2' at the top of the notebook to include your name(s)**.

Once that is done and you are happy with everything, you can then run the following cell to generate your PDF.

[52]:
```
# !jupyter nbconvert --to pdf mlp_week05_key.ipynb
```